

EXAMEN SUR TABLE - Reponses proposees

Codesign FPGA sur DE0-Nano

Auteur: etudiant Date: 2026-04-08

Q1 - Nombre de canaux analogiques acquis

Reponse : Le module capteurs_sol.vhd acquiert 8 conversions ADC par sequence (registres internes data0 a data7).

Justification courte:

- la variable channel est incrementee jusqu'a 1000 dans l'etat S4,
- le process de reception remplit bien data0 ... data7.

Nuance importante:

- l'interface de sortie expose seulement data0r ... data6r (7 canaux exportes vers le systeme Nios).

Q2 - Resolution effective

Reponse : La resolution effective disponible sur data0r ... data6r est 8 bits.

Explication:

- LTC2308 fournit 12 bits natifs.
- Dans S4, le code sort dataX(11 downto 4) : on tronque les 4 bits de poids faible (LSB).
- Impact: perte de finesse, pas de quantification multiplie par 16.

Q3 - Role de data_capture

Reponse : data_capture est le signal de declenchement d'une acquisition complete.

Evenement de depart:

- front montant de data_capture detecte en S0 (condition if data_capture = '1' then state <= S1).

Pourquoi S5 attend le retour a 0:

- pour imposer un mode "one-shot" (un declenchement par impulsion),
- pour eviter les redemarrages immediats si data_capture reste a 1.

Q4 - Role des etats S0 a S5

Reponse :

- S0: attente d'un nouveau trigger data_capture.
- S1: impulsion ADC_CONVST pour lancer la conversion ADC.
- S2: attente du temps de conversion (compteur wait_tick_cnt).
- S3: transfert SPI (envoi config canal + lecture bits ADC).
- S4: fin de canal, stockage, passage au canal suivant ou fin sequence.
- S5: maintien de data_ready et attente que data_capture retombe a 0.

Q5 - Calcul de CONVST_WAIT_CLOCK_NUM a 40 MHz

Reponse : Formule dans le code: CONVST_WAIT_CLOCK_NUM = ceil(1600 ns / CLOCK_DUR)

A 40 MHz:

- periode horloge: CLOCK_DUR = 25 ns,
- donc: CONVST_WAIT_CLOCK_NUM = ceil(1600/25) = 64.

Duree correspondante:

- 64 * 25 ns = 1600 ns = 1.6 us.

Phenomenes couverts:

- temps de conversion/latence interne ADC entre l'impulsion CONVST et des donnees valides en SPI.

Q6 - Pourquoi utiliser not clk pour ADC_SCK

Reponse : On inverse l'horloge pour respecter les marges temporelles (setup/hold).

Idee simple:

- ADC_SDI est mis a jour sur front montant de clk,

- ADC_SCK etant inverse, son front montant arrive plus tard (demi-periode),
- l'ADC echantillonne alors une donnee SDI stable.

Conclusion:

- meilleur alignement temporel du protocole SPI, moins de risque d'erreur de capture.

Q7 - Liste de sensibilité du process combinatoire

Reponse : Le process v2v_pr_8 calcule channel_config uniquement a partir de channel. Donc une sensibilité limitée a channel est correcte.

Si la table etait fausse/incomplete:

- en simulation: channel_config pourrait ne pas se mettre a jour correctement,
- consequence: ecart simulation vs synthese, bug fonctionnel difficile a debugger.

Q8 - Schema bloc codesign complet

Reponse : Diagramme d'architecture: allez voir le fichier schema_bloc_q8.mmd.

Signaux cles HW/SW (implementation reelle):

- **Commande calcul :** sensor_control et start_sl,
- **Operandes :** kp et kd (8 LSB utilises),
- **Resultat :** sensor_data6 (8 bits),
- **Statut :** sensor_status[0]=done, [1]=overflow, [2]=sensor_ready.

Q9 - Interface precise du bloc calculateur cable

Reponse : Definition proposee (4 operations via op_sel[1:0]).

Nom du port	Direction	Largeur	Role
clk	in	1	Horloge du bloc calculateur
reset_n	in	1	Reset actif a 0
start	in	1	Declenchement du calcul
op_sel	in	2	Selection operation: 00 add, 01 sub, 10 amplif, 11 attenu
data_ir	in	8	Operande i non signee
data_jr	in	8	Operande j non signee
result	out	8	Resultat sature sur 8 bits
overflow	out	1	1 si depassement ou underflow detecte
data_ready	out	1	1 quand resultat valide

Q10 - Detection de data_ready sans Avalon

Reponse : Mecanisme 1 (polling):

- le logiciel lit periodiquement sensor_status et teste le bit de disponibilite.
- dans notre implementation: bit2 = sensor_ready (capteurs), bit0 = done (calculateur).
- Avantage: tres simple a coder.
- Inconvenient: charge CPU + latence dependante de la periode de polling.

Mecanisme 2 (interruption IRQ):

- router data_ready vers un PIO avec interruption sur front montant.
- Avantage: faible latence, CPU libre entre deux acquisitions.
- Inconvenient: configuration plus complexe (ISR, masque IRQ, clear edge capture).

Q11 - Precautions overflow + exemple numerique

Reponse : Precautions a prendre:

- elargir les bus internes avant calcul (ex: 9 a 10 bits),
- distinguer non signe/signe selon operation,
- appliquer saturation a la sortie 8 bits (0..255),
- fournir un flag overflow.

Exemple complet:

- $\text{data_ir} = 240, \text{data_jr} = 100$ (8 bits non signes).
- Addition réelle: $240 + 100 = 340$.

Sans precaution (8 bits):

- $340 \bmod 256 = 84$ (overflow silencieux, faux resultat).

Avec bus elargi + saturation:

- calcul en 10 bits = 340,
- sortie 8 bits saturee = 255,
- overflow = 1.

Q12 - Amplification en logique cablee vs logicielle

Reponse : Pertinence de l'implementation cablee (gain > 1):

- Performance: tres bonne. Calcul en parallele, latence faible et deterministe.
- Ressources FPGA: cout en LUT/registres, mais faible si operation simple (shift/add).

Pertinence de l'implementation logicielle:

- Flexibilite: excellente. Gain modifiable rapidement, algorithmes evolutifs.
- Complexite de conception: plus simple a maintenir/corriger que du VHDL.

Conclusion pratique:

- gain fixe et rapide => hardware,
- gain adaptatif/complexe => software,
- approche hybride possible (pretraitement HW + ajustements SW).

Q13 - Etapes d'integration Quartus II 13.0

Reponse :

1. Creation du projet:

- File -> New Project Wizard,
- top-level entity: CONTROLE,
- device: Cyclone IV EP4CE22F17C6 (DE0-Nano).

2. Sources et hierarchie:

- ajouter: CONTROLE.vhd, capteurs_sol.vhd, calculateur_cable.vhd, clock_div2.vhd, pulse_gen.vhd,
- ajouter le systeme Qsys genere: nios_system_sdram.qsys puis generation HDL,
- verifier l'instanciation et les largeurs de ports.

3. Cible, horloge et brochage:

- declarer CLOCK_50 comme horloge,
- contraintes pin assignment minimales: SDRAM, ADC (CONVST/SCK/SDI/SDO), KEY, SW, LED,
- verifier les I/O standards (3.3V LVTTL),
- compiler completement le VHDL avec Quartus (jusqu'au .sof),
- programmer la carte avec Altera Monitor Program.

Q14 - Contrainte 50 MHz vs 40 MHz max capteurs_sol

Reponse : Probleme:

- la carte fournit CLOCK_50,
- capteurs_sol annonce max 40 MHz.

Solution retenue:

- faire tourner capteurs_sol a 25 MHz (division par 2 du 50 MHz), ou generer 40 MHz via PLL.

Impact sur CONVST_WAIT_CLOCK_NUM (cas 25 MHz):

- nouvelle periode: $\text{CLOCK_DUR} = 40 \text{ ns}$,
- $\text{CONVST_WAIT_CLOCK_NUM} = \text{ceil}(1600/40) = 40$.

Impact fonctionnel:

- temporisation physique conservee a 1.6 us,
- interface ADC dans la spec frequence, fiabilite amelioree.

Q15 - Implementation de l'architecture (code commente)

Reponse : Code VHDL commente: allez voir les fichiers `calculateur_cable.vhd` et `CONTROLE.vhd`.

Integration top-level (principe):

- relier `op_sel` a un PIO de commande logiciel,
- relier `data_ir/data_jr` a deux donnees capteurs ou registres SW,
- exposer `result/overflow/data_ready` vers PIOs lus par Nios II.

Q16 - Programme de validation logicielle (code commente)

Reponse : Programme C execute cote Nios II (fichier `software/validation_calculateur.c`).

Principe:

- le logiciel ecrit les operandes dans `kp` et `kd`,
- il configure `sensor_control` (choix operation),
- il envoie une impulsion `start_sl`,
- il attend `sensor_status[0]` (done),
- il lit le resultat dans `sensor_data6`,
- il compare avec un modele logiciel de reference.

Adresses utilisees (Qsys actuel):

- `sensor_control` = 0x1020
- `sensor_status` = 0x1030
- `sensor_data6` = 0x10A0
- `kp` = 0x10B0
- `start_sl` = 0x1110
- `kd` = 0x1120

Code complet: allez voir `software/validation_calculateur.c`. Test manuel interactif: allez voir `software/manual_calculateur.c`.

Resultat observe sur carte:

- affichage terminal: `=== Validation calculateur cable via Nios/Qsys ===`
- puis: `OK: no HW/SW mismatch.`

Interpretation:

- le calculateur VHDL et le modele C donnent les memes resultats sur tous les vecteurs testes,
- les LEDs allumees en fin de test correspondent a la derniere valeur ecrite (0xFF sur un cas sature), pas a une erreur.

Q17 - Demarche de debogage (data_ready bloque a 0)

Reponse : Ordre d'inspection recommande:

1) Signaux de base:

- `reset_n`, `clk`, `data_capture`, `State`.
- verifier que le FSM quitte S0 et atteint S4/S5.

2) Chemin SPI:

- `ADC_CONVST`, `spi_clk_enable`, `ADC_SCK`, `ADC_SD0`.
- verifier qu'il y a bien activite pendant une acquisition.

3) Validation de fin de sequence:

- condition de passage `channel` = "1000" en S4,
- affectation `data_ready` <= '1' puis retour S5->S0.

4) Causes probables a eliminer:

- reset maintenu actif,
- violation timing (50 MHz utilise sur un bloc limite a 40 MHz),
- mauvais brochage ADC (SDO/SCK/CONVST/SDI),
- bug de trigger (`data_capture` jamais vu a 1 puis 0),
- erreur de largeur/type (comparaison unsigned vs std_logic_vector).

Outils Quartus a utiliser:

- SignalTap II Logic Analyzer (prioritaire) pour voir les signaux internes en temps reel,

- Pin Planner pour verifier le brochage,
- TimeQuest Timing Analyzer pour valider les contraintes temporelles,
- RTL Viewer/Technology Map Viewer pour verifier la logique synthetisee.

Fin du document